

1. Tutorial básico: tablas, fórmulas y resaltado

Este capítulo presenta los elementos fundamentales del DSL mediante un ejemplo mínimo. Al terminar sabrá definir una tabla con columnas, añadir una columna calculada, condicionar el resaltado de celdas y generar el archivo de salida.

1.1 Qué vamos a construir

Queremos una tabla con cuatro columnas:

[le voy a poner los separadores de las columnas a la tabla.]

[También voy a resaltar en rojo los que tengan más de 80]

Nombre	Nota 1	Nota 2	Promedio
Ana	85	90	87.5
Luis	70	75	72.5
Elena	92	88	90.0
Carlos	60	65	62.5
Sofía	78	82	80.0
Pedro	95	91	93.0

Además:

- La columna **Promedio** no se escribe a mano: se calcula automáticamente como el promedio de las dos notas.
- Hay un **número fijo** (80, por ejemplo) almacenado en el libro.
- Los nombres de las personas cuyo promedio supera ese número aparecen **resaltados en rojo**.

1.2 Estructura del programa

El programa completo consta de tres partes, que escribiremos (en un solo archivo `tutorial-basico.lisp`¹) :

1: ¿que en qué carpeta lo creo?

1. Los **datos** de entrada (nombres y notas).
2. La **definición de la tabla** con su columna calculada y su regla de resaltado.
3. El **libro** que reúne todo y las instrucciones para generar la salida.

[Mmmm... aquí sugeriría hacerlo al revés... empezar por la tabla primero y después poner los datos... ¿no? La alternativa a eso es que expliques entonces por qué los datos de una tabla están en una lista de common lisp... Recuerda que el tutorial es para alguien que no tenga idea de nada de nada...]

[Deja ver cómo sigue la cosa, pero yo pensaría que la idea es explicar primero la estructura de la tabla (con el `def-table`, que supuestamente crea una función... ¿no?) y después cuando vayamos a llamar esa función, si no le pasamos datos, crearía una tabla sin datos, que no está mal, lo llenas a mano y ya, pero es más *tocão* cuando le pones datos... y ahí presentas los datos en forma de lista.]

[En cualquier caso, hay un problema más grande, que es que no me has dicho qué ficheros crear y desde dónde correr sbcl... por aquí falta una instrucción que sea crea el fichero en la misma carpeta donde están todas las cosas, y entonces... y sigues por ahí...]

1.3 Los datos

Los datos se guardan en una variable de Lisp con `defparameter`. Cada fila es una lista con cuatro (celdas²) (una por columna). Las (primeras tres filas son de prefijo^{Q1})³) : dos filas en blanco y una fila de encabezados. Las filas siguientes son los datos propiamente dichos.

2: será elementos, ¿no? ¿o qué significa que una lista tenga celdas?

Q1: ¿Qué es esto?

3: Antes de esto, en algún momento deberías explicarme qué son los prefijos esos

Listing 1.1: `tutorial-basico.lisp` — datos de entrada.

```
1 (defparameter *DATOS*  
2   '((" " " " " " "  
3     (" " " " "  
4     ("Nombre" "Nota 1" "Nota 2" "Promedio")  
5     ("Ana"      "85" "90" " ")))
```

```

6      ("Luis"      "70" "75" "")
7      ("Elena"     "92" "88" "")
8      ("Carlos"    "60" "65" "")
9      ("Sofia"     "78" "82" "")
10     ("Pedro"     "95" "91" ""))

```

Observe que la columna Promedio se (deja vacía ⁴) en los datos, porque su valor lo calculará (el DSL automáticamente⁵).

4: Por algún motivo aquí no salen las comillas :-/... y lo que veo es una lista vacía en Common Lisp, que me confunde un poquito...

Formato de las celdas vacías

Las celdas que corresponden a columnas calculadas se marcan con (⁶) (cadena vacía). (El generador^{Q2}) las sobrescribe (con la fórmula correspondiente^{Q3}).

5: ¿el DSL o el excel cuando se cree después? Aquí sería bueno separar el DSL del producto final...

TRANSICIÓN A LO QUE SIGUE.

1.4 Definir la tabla

La macro `def-table` declara (una plantilla de tabla^{Q4}). Le damos un nombre y la lista de columnas. Cada columna se escribe como un par (nombre ^{Etiqueta}), donde nombre es el (identificador interno^{Q5}) (un símbolo del DSL) y ^{Etiqueta} es el texto que se mostrará (como encabezado⁷).

6: Decididamente esta cadena vacía no se ve. Hay que buscar una forma que de que vean.

Q2: ¿Qué es esto?

Q3: ¿Qué es esto?

Aquí sería bueno poner un ejemplito de eso de los encabezados y los identificadores internos. Cuando me refiero a un ejemplito, no me refiero a la tabla que viene ahí a continuación sino a algo explicado con palabras... para entender la idea, y después poderlo *entender* en el ejemplo.

Listing 1.2: Definición de la tabla.

```

1  (def-table tabla-notas
2    ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3    :cell-height 1
4    :cell-width 1)

```

[Yo sé que en el tutorial no me has mandado a hacerlo, pero en common lisp, la idea es ir creando las cosas y probándolas inmediatamente... si trato de ejecutar ese fragmento de código, me da este error]

The variable TABLA-NOTAS is unbound.

[Condition of type UNBOUND-VARIABLE]

Por ahora la tabla solo declara las columnas, sin fórmulas ni resaltados. [Aquí yo sugeriría decir que si creamos un libro con esta tabla solo estarían los encabezados... y ya, más nada.] A continuación añadiremos esas características.

[Mmm.... a lo mejor conviene que el tutorial tenga tres partes: una primera en la que solo se crean los encabezados de la tabla, ni siquiera con datos. y con eso, crear un libro y exportarlo y todo. Incluso, mostrar el resultado del excel, poniendo unas foticos. Ese sería el tutorial ultra básico para confirmar que el sistema funciona. Después, vamos a ponerle datos, y después las otras dos cosas.]

[Para eso antes de tirar una línea de código debes explicar lo que se puede hacer con esto: crear tablas, que se ponen en hojas, que a su vez, se ponen en libros. Todo eso se hace en lisp. Después, esos *libros* se exportan a alguno de los backend disponibles, y después, puedes experimentar con esas tablas, hojas y libros, en la arquitectura en la que se hayan creado.]

1.5 Columna calculada

Queremos (que promedio⁸) (se calcule solo⁹) :

$$\text{promedio} = \frac{\text{nota1} + \text{nota2}}{2}$$

[¿qué son nota1 y nota2?]

En lenguaje natural diríamos: “(para cada fila¹⁰) , promedia nota1 y nota2”. El DSL ofrece la forma `promedio` que hace exactamente eso: recibe (una lista de columnas^{Q6}) , genera la suma y la divide entre la cantidad de columnas automáticamente.

Q4: ¿Qué es esto?

Q5: ¿Qué es esto?

7: ¿cómo encabezado de qué?

8: ¿qué es promedio? ¿o te refieres a los elementos de la columna promedio?

Listing 1.3: Tabla con columna calculada.

```
1 (def-table tabla-notas
2   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((promedio (promedio (col nota1) (col nota2)))))
```

[Si intento ejecutar eso me da el mismo error de arriba... Que tiene sentido, porque no en lo que he hecho no está definido `def-table`.]

Cada pieza significa:

- `(col nota1)` — toma el valor de la columna `nota1` (en la fila actual¹¹)
- `(promedio (col nota1) (col nota2))` — suma `nota1` y `nota2`, y divide entre 2 (porque recibe dos argumentos).

9: ¿qué significa esto?

¿Cómo sabe promedio entre cuánto dividir?

`promedio` cuenta automáticamente los argumentos que recibe. Con `(promedio (col n1) (col n2))` genera $((n1+n2)/2)$; si recibe tres columnas genera $((n1+n2+n3)/3)$. No hace falta indicar el divisor.

[Ahora mismo te lo tengo que creer... Si tuviéramos la variante de crear el libro con esa hoja y esa tabla, pudiéramos comprobarlo de verdad.]

Este sería un buen momento para poner un cuadrito chulo de eso 8-) en el que comentas qué otras posibles funciones se pueden usar en lugar de `promedio`... Por ejemplo, si quiero la suma o si quiero comprobar si `nota1` es mayor que `nota2`.

TRANSICIÓN A LO QUE SIGUE.

1.6 Resaltado condicional

Ahora queremos que los nombres de las personas cuyo promedio supere el mínimo aparezcan en rojo. En lenguaje natural: “si el promedio de esta fila es mayor que `nota-minima`, colorea el nombre”.

La opción `:render` de `def-table` acepta una forma `conditional-rendering` que especifica la condición y las columnas a colorear.

Listing 1.4: Tabla con columna calculada y resaltado.

```
1 (def-table tabla-notas
2   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((promedio (promedio (col nota1) (col nota2))))
7   :render
```

```

8      (conditional-rendering
9        :condition (gt (col promedio) (param nota-
10                       minima)))
        :target-columns (nombre)))

```

[:-o... buenísimo esto :-o... lo entendí a la primera :-).]

Al leer la condición en voz alta:

- `gt` significa “mayor que” (*greater than*).
- `(col promedio)` es el promedio de la fila actual.
- `(param nota-minima)` es el valor fijo que definiremos.
- `:target-columns (nombre)` indica qué columna se colorea cuando la condición se cumple.

[Aquí vendría el mismo comentario que arriba, comenta por aquí (quizás en un cuadrito lindo) qué otras opciones existen para las comparaciones y cómo se pueden usar los operadores lógicos de conjunción, disyunción y negación.]

[Y lo otro que sería bueno comentar aquí es qué hacer cuando quieres que se resalten más de una columna, o cuando quieres tener dos condicionales...]

[Bueno, en realidad, queda un problema colgao que es el `(param nota-minima)` ese... del que no has dicho... la buena noticia es que eso se puede resolver cambiando el orden del tutorial...]

TRANSICIÓN A LO QUE SIGUE.

1.7 El parámetro y la generación

(Con la tabla definida, solo falta ensamblar el libro¹²) , proporcionar los parámetros y (lanzar la generación^{Q7}) .

(Los parámetros¹³) se pasan (al instanciar la tabla con `tabla`¹⁴) mediante la clave `:params`. Cada parámetro es un par (nombre valor).

Listing 1.5: Libro completo con parámetros.

```

1 (libro tutorial-basico
2   :filename "Tutorial_Basico.xlsx"
3   :hojas (list
4     (hoja "Notas"

```

10: ¿qué significa esto?

Q6: ¿Qué es esto?

11: ¿qué es la fila actual?

12: esto sería bueno cambiarlo con la nueva idea, de hacerlo todo paso a paso... incluso, antes de los parámetros, puedes presentar cómo insertar solo un texto, y después dices que puede ser un parámetro.

```

5      (tabla tabla-notas
6        :data *DATOS*
7        :params ((nota-minima 80))))))
8
9  (xl-generate tutorial-basico "tutorial-basico.py")
10 (xl-run-generated "tutorial-basico.py")

```

[Aquí me sigue pareciendo una mala idea que el libro tenga un :filename. Eso se debería cambiar... revisa los comentarios que te puse sobre eso en el otro documento.]

El concepto de parámetro

Un `param` es un valor fijo con nombre que se escribe una sola vez y (se referencia desde cualquier expresión¹⁵) con `(param nombre)`. Es la manera que tiene el DSL de representar constantes que puedan cambiar entre ejecuciones sin tocar las fórmulas. En este ejemplo (`param nota-minima`) vale 80.

En la implementación concreta del generador a Excel, (los parámetros se traducen a celdas fijas de la hoja (por ejemplo `$F$1`)¹⁶) para que las fórmulas puedan referenciarlos con referencias absolutas. (El backend de Python^{Q8}) los escribe directamente en (esas celdas¹⁷) , (sin mezclarlos con las filas de datos de la tabla¹⁸) .

TRANSICIÓN A LO QUE SIGUE.

1.8 El programa completo

A continuación se muestra el contenido íntegro del archivo `tutorial-basico.lisp`:

Listing 1.6: `tutorial-basico.lisp` — programa completo.

```

1  (load (merge-pathnames "dsl-directo.lisp" *load-
   true-name*))
2
3  (defparameter *DATOS*
4    '((" " " " " ")
5      (" " " " " ")
6      ("Nombre" "Nota 1" "Nota 2" "Promedio")
7      ("Ana" "85" "90" " ")
8      ("Luis" "70" "75" " ")
9      ("Elena" "92" "88" " ")
10     ("Carlos" "60" "65" " "))

```

```

11     ("Sofia"      "78" "82" "")
12     ("Pedro"      "95" "91" "")))
13
14 (def-table tabla-notas
15   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
16   :cell-height 1
17   :cell-width 1
18   :computed
19   ((promedio (promedio (col nota1) (col nota2))))
20   :render
21   (conditional-rendering
22     :condition (gt (col promedio) (param nota-
23                     minima))
24     :target-columns (nombre)))
25
26 (libro tutorial-basico
27   :filename "Tutorial_Basico.xlsx"
28   :hojas (list
29     (hoja "Notas"
30       (tabla tabla-notas
31         :data *DATOS*
32         :params ((nota-minima 80)))))
33
34 (xl-generate tutorial-basico "tutorial-basico.py")
35 (xl-run-generated "tutorial-basico.py")

```

[En el render tuve una duda... quise que me resaltara las columnas nombre y promedio... pero eso no pasó... lo que hice fue ponerle el nombre de la columna promedio, pero eso no funcionó... ¿tengo que poner dos formatos condicionales?]

1.9 Ejecución

Para ejecutar el tutorial:

```

cd tutorial/
sbcl --script tutorial-basico.lisp

```

Esto cargará el DSL, construirá el AST, generará `tutorial-basico.py` y lo ejecutará para crear `Tutorial_Basico.xlsx`.

1.10 Resultado esperado

El archivo `Tutorial_Basico.xlsx` contiene una hoja llamada “Notas” con la siguiente estructura:

Rango	Contenido
Filas 3–9, columnas A–D	Tabla con 6 personas (fila 3: encabezados)
Columna D, filas 4–9	Promedio calculado con fórmula
Columna A, filas 4–9	Nombres en rojo si promedio > 80

Con los datos de ejemplo, los nombres de Ana, Elena, Sofía y Pedro aparecen resaltados en rojo porque su promedio supera 80. Luis y Carlos quedan con formato normal.

1.11 Resumen de formas utilizadas

Forma	Para qué sirve
<code>(def-table nombre (cols...) opts...)</code>	Define una plantilla de tabla
<code>:computed ((col expr))</code>	Columna calculada con una expresión
<code>(conditional-rendering :condition :target-columns)</code>	Regla de resaltado condicional
<code>(promedio col1 col2 ...)</code>	Promedia las columnas indicadas
<code>(gt a b)</code>	Verdadero si a es mayor que b
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(param nombre)</code>	Valor de un parámetro fijo
<code>:params ((nombre valor)...))</code>	Define parámetros al instanciar la tabla
<code>(libro nombre ...)</code>	Define el libro completo
<code>(hoja nombre tablas ...)</code>	Define una hoja
<code>(tabla plantilla :data :params)</code>	Instancia una tabla con datos y parámetros
<code>(xl-generate wb archivo)</code>	Genera el script Python
<code>(xl-run-generated archivo)</code>	Ejecuta el script y produce el .xlsx